

G.A.N.E NAV — APK / TWA Generation Guide

This document explains how to package G.A.N.E NAV as a native Android APK using **Trusted Web Activity (TWA)**, which wraps the PWA in a native Android shell with full Chrome rendering.

What is TWA?

A **Trusted Web Activity** is an Android app that displays a full-screen Chrome browser tab without any browser UI. It provides a native app experience while running the actual PWA. TWA apps:

- Appear in the Play Store like any other app
 - Use Chrome's rendering engine (no WebView limitations)
 - Support push notifications, offline mode, and all PWA features
 - Require Digital Asset Links verification (proves you own the domain)
-

Prerequisites

Requirement	Details
Published PWA	G.A.N.E NAV must be published and accessible via HTTPS
Valid manifest.json	Already created at <code>/client/public/manifest.json</code>
Service Worker	Already created at <code>/client/public/sw.js</code>
Android Studio	For building the APK (or use Bubblewrap CLI)
Java JDK 11+	Required for Android build tools
Google Play Console	For publishing to Play Store (\$25 one-time fee)

Method 1: Bubblewrap CLI (Recommended)

Bubblewrap is Google's official tool for generating TWA projects from PWAs.

Step 1: Install Bubblewrap

Bash

```
npm install -g @nicolo-ribaudo/bubblewrap
```

Step 2: Initialize the TWA Project

Bash

```
mkdir gane-nav-twa && cd gane-nav-twa  
bubblewrap init --manifest https://YOUR_DOMAIN/manifest.json
```

This will prompt you for:

- **Application ID:** com.gane.nav
- **App Name:** G.A.N.E NAV
- **Launcher Name:** G.A.N.E
- **Display Mode:** standalone
- **Status Bar Color:** #0A0F1C
- **Splash Screen Color:** #0A0F1C
- **Icon:** Uses the 512x512 icon from manifest.json
- **Signing Key:** Creates a new keystore (save this securely!)

Step 3: Build the APK

Bash

```
bubblewrap build
```

This generates:

- app-release-signed.apk — Ready to install or upload to Play Store
- app-release-bundle.aab — Android App Bundle for Play Store

Step 4: Digital Asset Links

Create a file at `/.well-known/assetlinks.json` on your domain:

JSON

```
[{
  "relation": ["delegate_permission/common.handle_all_urls"],
  "target": {
    "namespace": "android_app",
    "package_name": "com.gane.nav",
    "sha256_cert_fingerprints": [
      "YOUR_SHA256_FINGERPRINT"
    ]
  }
}]
```

Get your fingerprint:

Bash

```
keytool -list -v -keystore YOUR_KEYSTORE.jks -alias YOUR_ALIAS
```

Method 2: PWABuilder (No-Code)

1. Go to [PWABuilder.com](https://pwabuilder.com)
2. Enter your published PWA URL
3. Click "Package for stores"
4. Select "Android"
5. Configure options (package name, signing key)
6. Download the generated APK/AAB

Method 3: Android Studio (Manual)

Step 1: Create a New Android Project

- Template: "Empty Activity"
- Package: `com.gane.nav`
- Minimum SDK: API 21 (Android 5.0)

Step 2: Add TWA Dependencies

In `build.gradle` :

Plain Text

```
dependencies {  
    implementation 'com.google.androidbrowserhelper:androidbrowserhelper:2.5.0'  
}
```

Step 3: Configure the Activity

In `AndroidManifest.xml` :

XML

```
<activity android:name="com.google.androidbrowserhelper.trusted.LauncherActivit  
    android:exported="true">  
    <meta-data android:name="android.support.customtabs.trusted.DEFAULT_URL"  
        android:value="https://YOUR_DOMAIN" />  
    <intent-filter>  
        <action android:name="android.intent.action.MAIN" />  
        <category android:name="android.intent.category.LAUNCHER" />  
    </intent-filter>  
    <intent-filter android:autoVerify="true">  
        <action android:name="android.intent.action.VIEW"/>  
        <category android:name="android.intent.category.DEFAULT" />  
        <category android:name="android.intent.category.BROWSABLE"/>  
        <data android:scheme="https" android:host="YOUR_DOMAIN"/>  
    </intent-filter>  
</activity>
```

Step 4: Build and Sign

Bash

```
./gradlew assembleRelease
```

iOS: Add to Home Screen

iOS does not support TWA, but G.A.N.E NAV works as a PWA on iOS Safari:

1. Open the published URL in Safari
2. Tap the Share button
3. Select "Add to Home Screen"
4. The app runs in standalone mode (no browser chrome)

The manifest.json and meta tags already configure:

- `apple-mobile-web-app-capable` : Standalone mode
 - `apple-mobile-web-app-status-bar-style` : Black translucent
 - `apple-touch-icon` : App icon
-

Desktop: Install as App

On Chrome/Edge desktop:

1. Visit the published URL
 2. Click the install icon in the address bar (or use the PWA install prompt)
 3. The app installs as a desktop application
-

Distribution Channels Summary

Platform	Method	Store Distribution
Android	TWA (Bubblewrap/PWABuilder)	Google Play Store
iOS	PWA (Add to Home Screen)	No store needed
Windows	PWA Install	Microsoft Store (optional)
macOS	PWA Install	No store needed
Linux	PWA Install	No store needed
ChromeOS	PWA Install	Chrome Web Store (optional)

Testing the APK

1. **Local testing:** `adb install app-release-signed.apk`
2. **Emulator:** Use Android Studio's AVD Manager
3. **Real device:** Enable USB debugging, connect device, run `adb install`

Test checklist:

- ☐ App launches in full-screen (no browser UI)

- ☐ Navigation works correctly
 - ☐ Offline mode works
 - ☐ Push notifications work
 - ☐ GPS/location works
 - ☐ Service worker caches properly
 - ☐ Back button behavior is correct
-

Play Store Publishing

1. Create a Google Play Console account (\$25)
 2. Create a new app listing
 3. Upload the AAB file
 4. Fill in store listing (description, screenshots, etc.)
 5. Set up content rating
 6. Set pricing (free)
 7. Submit for review
-

Keystore Security

CRITICAL: Keep your signing keystore file safe. If lost, you cannot update the app on the Play Store.

Bash

```
# Backup keystore
cp android.keystore /secure/backup/location/

# Store password securely (never commit to git)
echo "KEYSTORE_PASSWORD=..." >> .env.local
```